Práctica N°2: Sumador de 3 bits usando pantalla OLED en placa ZedBoard

Índice

1. Objetivos **2**
- 1.1. Objetivo general 2
- 1.2. Objetivos específicos 2

2. Descripción del código **3**
- 2.1. Módulo 1: `adder3` (sumador 3+3 bits) 3
- 2.2. Módulo 2: `oled_ex` (demo OLED + visualización de A, B y A+B) 4
- 2.3. Módulo 3: `oled_ctrl` (top: inicialización, multiplexado y ruteo de A, B) 17

3. Asignación de pines y señales **22**

4. Resultados **23**

1. Objetivos

1.1. Objetivo general

Implementar en VHDL un **sumador de 3 bits** sobre la plataforma *ZedBoard*, **reutilizando la base existente de la OLED** (SSD1306) con cambios mínimos al proyecto, de modo que los operandos **A** y **B** se ingresen por interruptores y el resultado **A + B** se visualice en la pantalla OLED.

1.2. Objetivos específicos

1. **Encapsular la aritmética:** diseñar e integrar un bloque `adder3` que sume dos vectores de 3 bits y entregue un resultado de 4 bits, evitando conflictos de librerías en el código legado.

2. **Exposición de señales:** añadir puertos `a[2:0]` y `b[2:0]` en el nivel superior (`oled_ctrl`) y enrutar estas señales hasta `oled_ex`.

3. **Actualización mínima del demo OLED:** modificar exclusivamente la rama de contenido en `oled_ex` para construir y mostrar la cadena `.ᴬ=_ B=_ A+B=_"` en formato hexadecimal (1 dígito por operando y 1 dígito para el resultado).

4. **Soporte de caracteres:** verificar y, de ser necesario, añadir en `ascii_rom` los glifos de `'+'` (0x2B) y `'='` (0x3D) sin alterar el resto de la ROM.

5. **Asignación de pines:** mapear los bits de `a` y `b` a los *switches* físicos de la ZedBoard mediante el archivo de restricciones, manteniendo intacta la lógica de inicialización y SPI.

6. **Verificación funcional:** comprobar en hardware que para todos los casos $A, B \in [0, 7]$ la OLED muestre correctamente A, B y $A + B$ (0–14), incluyendo casos de "overflow" (por ejemplo $7 + 7 = 14$).

7. **Criterios de éxito:** la visualización debe actualizarse sin *glitches*, el proyecto debe sintetizar sin advertencias críticas y no deben introducirse modificaciones innecesarias fuera de los puntos descritos.

2. Descripción del código

Para esta práctica se **reutilizan exactamente los mismos archivos de inicialización y control SPI** de la práctica anterior (`oled_init.vhd`, `spi_ctrl.vhd`, `delay.vhd` y `ascii_rom.vhd`), de forma que **no se tocan sus máquinas de estados ni sus temporizaciones**. El cambio es mínimo y se concentra en **tres puntos**: (1) en el *top module* (`oled_ctrl.vhd`) se exponen los puertos `a[2:0]` y `b[2:0]` para llevar los switches a la jerarquía; (2) se **crea** un bloque nuevo `adder3.vhd` encargado exclusivamente de la suma en 3+3 bits (aislando las librerías aritméticas); y (3) se **modifica** `oled_ex.vhd` únicamente en la rama que arma el texto para la OLED, sustituyendo el mensaje de demo por la visualización de `A, B` y `A+B`. Este enfoque minimiza el riesgo de efectos colaterales y obliga a una separación clara de responsabilidades (inicialización, SPI, aritmética y presentación).

2.1. Módulo 1: `adder3` (sumador 3+3 bits)

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity adder3 is
  port (
    a    : in  std_logic_vector(2 downto 0);
    b    : in  std_logic_vector(2 downto 0);
    sum4 : out std_logic_vector(3 downto 0)   -- resultado 0..14
  );
end entity;

architecture rtl of adder3 is
begin
  -- Suma desambiguada dentro del bloque (solo numeric_std aqu )
  sum4 <= std_logic_vector( unsigned('0' & a) + unsigned('0' & b) );
end architecture;
```

Listing 1: Bloque de suma aislado `adder3.vhd` (3+3 bits → 4 bits).

¿Qué hace y por qué así?

- **Entradas y salida**. Recibe dos operandos de 3 bits (A y B) y entrega un resultado de 4 bits `sum4` $\in [0, 14]$. El bit extra cubre el posible acarreo en $7 + 7$.
- **Extensión de tamaño**. La expresión `'0' & a` (y análoga para `b`) antepone un cero para extender cada operando a 4 bits antes de sumar.
- **Aritmética tipada**. Se realiza el *cast* a `unsigned` y se usa `numeric_std`, lo que **evita ambigüedades de sobrecarga** del operador `+`. Luego se convierte el resultado a `std_logic_vector` para compatibilizar con el resto del diseño.
- **Encapsulación**. Al aislar la suma aquí, **no se modifican** los paquetes usados en archivos heredados (que emplean `std_logic_arith`/`std_logic_unsigned`). Así prevenimos conflictos sin refactorizar el legado.
- **Uso previsto**. Este bloque se instancia en `oled_ex.vhd`, y su salida `sum4` se convierte a ASCII (hex) para mostrar `A`, `B` y `A+B` en la OLED.

2.2. Módulo 2: `oled_ex` (demo OLED + visualización de A, B y A+B)

```
1  --
2  -- Written by Ryan Kim, Digilent Inc.
3  -- Modified by Michael Mattioli
4  --
5  -- Description: Demo for the OLED display. First
       displays the alphabet for ~4 seconds and then
6  -- clears the display, waits for a ~1 second and then
       displays "Hello world!".
7  --
8
9  library ieee;
10 use ieee.std_logic_1164.all;
11 use ieee.std_logic_arith.all;
12 use ieee.std_logic_unsigned.all;
13
14 entity oled_ex is
15     port (  clk         : in std_logic; -- System clock
16             rst         : in std_logic; -- Global
                   synchronous reset
17             en          : in std_logic; -- Block enable
                   pin
```

```
18              sdout       : out std_logic; -- SPI data out
19              oled_sclk   : out std_logic; -- SPI clock
20              oled_dc     : out std_logic; -- Data/Command
                    controller
21              fin         : out std_logic; -- Finish flag
                   for block
22              a    : in std_logic_vector(2 downto 0); -- 3
                    bits
23              b      : in std_logic_vector(2 downto 0));
                   -- 3 bits
24  end oled_ex;
25
26  architecture behavioral of oled_ex is
27
28      -- SPI controller
29      component spi_ctrl
30          port (  clk         : in std_logic;
31                  rst         : in std_logic;
32                  en          : in std_logic;
33                  sdata       : in std_logic_vector (7
                       downto 0);
34                  sdout       : out std_logic;
35                  oled_sclk   : out std_logic;
36                  fin         : out std_logic);
37      end component;
38
39      -- delay controller
40      component delay
41          port (  clk         : in std_logic;
42                  rst         : in std_logic;
43                  delay_ms    : in std_logic_vector (11
                       downto 0);
44                  delay_en    : in std_logic;
45                  delay_fin   : out std_logic);
46      end component;
47
48      -- character library, latency = 1
49      component ascii_rom
50          port (  clk    : in std_logic; -- System clock
51                  addr   : in std_logic_vector (10 downto
                       0); -- First 8 bits is the ASCII
                       value of the character, the last 3
                       bits are the parts of the char
```

```
52                dout    : out std_logic_vector (7 downto
                     0)); -- Data byte out
53      end component;
54
55      -- States for state machine
56      type states is (Idle,
57                      ClearDC,
58                      SetPage,
59                      PageNum,
60                      LeftColumn1,
61                      LeftColumn2,
62                      SetDC,
63                      Alphabet,
64                      Wait1,
65                      ClearScreen,
66                      Wait2,
67                      HelloWorldScreen,
68                      UpdateScreen,
69                      SendChar1,
70                      SendChar2,
71                      SendChar3,
72                      SendChar4,
73                      SendChar5,
74                      SendChar6,
75                      SendChar7,
76                      SendChar8,
77                      ReadMem,
78                      ReadMem2,
79                      Done,
80                      Transition1,
81                      Transition2,
82                      Transition3,
83                      Transition4,
84                      Transition5);
85
86      type oled_mem is array (0 to 3, 0 to 15) of
           std_logic_vector (7 downto 0);
87
88      -- Variable that contains what the screen will be
           after the next UpdateScreen state
89      signal current_screen : oled_mem;
90
91      signal sum4 : std_logic_vector(3 downto 0);
```

```
92
93      -- Convierte 0..15 a '0'..'9','A'..'F'
94  function nibble_to_ascii(n : std_logic_vector(3 downto
        0))
95    return std_logic_vector is
96      begin
97        case n is
98          when "0000" => return x"30"; -- '0'
99          when "0001" => return x"31";
100         when "0010" => return x"32";
101         when "0011" => return x"33";
102         when "0100" => return x"34";
103         when "0101" => return x"35";
104         when "0110" => return x"36";
105         when "0111" => return x"37";
106         when "1000" => return x"38";
107         when "1001" => return x"39";
108         when "1010" => return x"41"; -- 'A'
109         when "1011" => return x"42";
110         when "1100" => return x"43";
111         when "1101" => return x"44";
112         when "1110" => return x"45";
113         when others => return x"46"; -- 'F'
114       end case;
115     end function;
116
117     -- Constant that contains the screen filled with the
            Alphabet and numbers
118     constant alphabet_screen : oled_mem := ((x"41", x"42
            ", x"43", x"44", x"45", x"46", x"47", x"48", x"49
            ", x"4A", x"4B", x"4C", x"4D", x"4E", x"4F", x"50
            "),
119     (x"51", x"52", x"53", x"54", x"55", x"56", x"57", x"
            58", x"59", x"5A", x"61", x"62", x"63", x"64", x"
            65", x"66"),
120     (x"67", x"68", x"69", x"6A", x"6B", x"6C", x"6D", x"
            6E", x"6F", x"70", x"71", x"72", x"73", x"74", x"
            75", x"76"),
121     (x"77", x"78", x"79", x"7A", x"30", x"31", x"32", x"
            33", x"34", x"35", x"36", x"37", x"38", x"39", x"
            7F", x"7F"));
122
```

```
123     -- Constant that fills the screen with blank (spaces
           ) entries
124     constant clear_screen : oled_mem := (   (x"20", x"20
           ", x"20", x"20", x"20", x"20", x"20", x"20", x"20
           ", x"20", x"20", x"20", x"20", x"20", x"20", x"20
           "),
125     (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20"),
126     (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20"),
127     (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20"));
128
129     -- Constant that holds "Hello world!"
130     constant hello_world_screen : oled_mem := ( (x"48",
           x"65", x"6c", x"6c", x"6f", x"20", x"77", x"6f",
           x"72", x"6c", x"64", x"21", x"20", x"20", x"20",
           x"20"),
131     (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20"),
132     (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20"),
133     (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20", x"20", x"20", x"20", x"20", x"20", x"
           20", x"20"));
134
135     -- Current overall state of the state machine
136     signal current_state : states := Idle;
137
138     -- State to go to after the SPI transmission is
           finished
139     signal after_state : states;
140
141     -- State to go to after the set page sequence
142     signal after_page_state : states;
143
144     -- State to go to after sending the character
           sequence
```

```
145     signal after_char_state : states;
146 
147     -- State to go to after the UpdateScreen is finished
148     signal after_update_state : states;
149 
150     -- Contains the value to be outputted to oled_dc
151     signal temp_dc : std_logic := '0';
152 
153     -- Used in the Delay controller block
154     signal temp_delay_ms : std_logic_vector (11 downto
        0); -- Amount of ms to delay
155     signal temp_delay_en : std_logic := '0'; -- Enable
        signal for the Delay block
156     signal temp_delay_fin : std_logic; -- Finish signal
        for the Delay block
157 
158     -- Used in the SPI controller block
159     signal temp_spi_en : std_logic := '0'; -- Enable
        signal for the SPI block
160     signal temp_sdata : std_logic_vector (7 downto 0) :=
         (others => '0'); -- Data to be sent out on SPI
161     signal temp_spi_fin : std_logic; -- Finish signal
        for the SPI block
162 
163     signal temp_char : std_logic_vector (7 downto 0) :=
        (others => '0'); -- Contains ASCII value for
        character
164     signal temp_addr : std_logic_vector (10 downto 0) :=
         (others => '0'); -- Contains address to byte
        needed in memory
165     signal temp_dout : std_logic_vector (7 downto 0); --
         Contains byte outputted from memory
166     signal temp_page : std_logic_vector (1 downto 0) :=
        (others => '0'); -- Current page
167     signal temp_index : integer range 0 to 15 := 0; --
        Current character on page
168 
169 begin
170 
171     oled_dc <= temp_dc;
172 
173     -- "Example" finish flag only high when in done
        state
```

```
174     fin <= '1' when current_state = Done else '0';
175
176     -- Instantiate SPI controller
177     spi_comp: spi_ctrl port map (   clk => clk,
178                                     rst => rst,
179                                     en => temp_spi_en,
180                                     sdata => temp_sdata,
181                                     sdout => sdout,
182                                     oled_sclk =>
                                           oled_sclk,
183                                     fin => temp_spi_fin)
                                           ;
184
185     -- Instantiate delay
186     delay_comp: delay port map (clk => clk,
187                                 rst => rst,
188                                 delay_ms =>
                                       temp_delay_ms,
189                                 delay_en =>
                                       temp_delay_en,
190                                 delay_fin =>
                                       temp_delay_fin);
191
192     -- Instantiate ASCII character library
193     char_lib_comp : ascii_rom port map (clk => clk,
194                                         addr =>
                                               temp_addr,
195                                         dout =>
                                               temp_dout);
196
197     u_adder3 : entity work.adder3 port map (
198                 a    => a,
199                 b    => b,
200                 sum4 => sum4);
201
202     process (clk)
203     begin
204         if rising_edge(clk) then
205             case current_state is
206                 -- Idle until en pulled high than
                       intialize Page to 0 and go to state
                       alphabet afterwards
207                 when Idle =>
```

```
208                     if en = '1' then
209                         current_state <= ClearDC;
210                         after_page_state <= Alphabet;
211                         temp_page <= "00";
212                     end if;
213                 -- Set current_screen to constant
                       alphabet_screen and update the screen
                       ; go to state Wait1 afterwards
214                 when Alphabet =>
215                     current_screen <= alphabet_screen;
216                     current_state <= UpdateScreen;
217                     after_update_state <= Wait1;
218                 -- Wait 4ms and go to ClearScreen
219                 when Wait1 =>
220                     temp_delay_ms <= "111110100000"; --
                           4000
221                     after_state <= ClearScreen;
222                     current_state <= Transition3; --
                           Transition3 = delay transition
                           states
223                 -- Set current_screen to constant
                       clear_screen and update the screen;
                       go to state Wait2 afterwards
224                 when ClearScreen =>
225                     current_screen <= clear_screen;
226                     after_update_state <= Wait2;
227                     current_state <= UpdateScreen;
228                 -- Wait 1ms and go to HelloWorldScreen
229                 when Wait2 =>
230                     temp_delay_ms <= "001111101000"; --
                           1000
231                     after_state <= HelloWorldScreen;
232                     current_state <= Transition3; --
                           Transition3 = delay transition
                           states
233                 -- Set currentScreen to constant
                       hello_world_screen and update the
                       screen; go to state Done afterwards
234                 when HelloWorldScreen =>
235                     -- Limpiar la pantalla base
236                     current_screen <= clear_screen;
237
```

```
238                     -- Linea 0: "A=H B=H A+B=H" (H =
                           digito HEX)
239                     current_screen(0, 0)  <= x"41";
                                              -- 'A'
240                     current_screen(0, 1)  <= x"3D";
                                              -- '='
241                     current_screen(0, 2)  <=
                           nibble_to_ascii('0' & a); -- A (
                           hex)
242
243                     current_screen(0, 3)  <= x"20";
                                              -- espacio
244                     current_screen(0, 4)  <= x"42";
                                              -- 'B'
245                     current_screen(0, 5)  <= x"3D";
                                              -- '='
246                     current_screen(0, 6)  <=
                           nibble_to_ascii('0' & b); -- B (
                           hex)
247
248                     current_screen(0, 7)  <= x"20";
                                              -- espacio
249                     current_screen(0, 8)  <= x"41";
                                              -- 'A'
250                     current_screen(0, 9)  <= x"2B";
                                              -- '+'
251                     current_screen(0,10)  <= x"42";
                                              -- 'B'
252                     current_screen(0,11)  <= x"3D";
                                              -- '='
253                     current_screen(0,12)  <=
                           nibble_to_ascii(sum4);     -- A+B
                           (hex)
254
255                     -- Mantener el flujo original
256                     after_update_state <= Done;
257                     current_state      <= UpdateScreen;
258
259                 -- Do nothing until en is deassertted
                       and then current_state is Idle
260                 when Done             =>
261                     if en = '0' then
262                         current_state <= Idle;
```

```
263                     end if;
264
265                 -- UpdateScreen State
266                 -- 1. Gets ASCII value from
                       current_screen at the current page
                       and the current spot
267                 --    of the page
268                 -- 2. If on the last character of the
                       page transition update the page
                       number, if on
269                 --    the last page(3) then the
                       updateScreen go to "
                       after_update_state" after
270                 when UpdateScreen =>
271                     temp_char <= current_screen(
                           conv_integer(temp_page),
                           temp_index);
272                     if temp_index = 15 then
273                         temp_index <= 0;
274                         temp_page <= temp_page + 1;
275                         after_char_state <= ClearDC;
276                         if temp_page = "11" then
277                             after_page_state <=
                                   after_update_state;
278                         else
279                             after_page_state <=
                                   UpdateScreen;
280                         end if;
281                     else
282                         temp_index <= temp_index + 1;
283                         after_char_state <= UpdateScreen
                               ;
284                     end if;
285                     current_state <= SendChar1;
286
287                 -- Update Page states
288                 -- 1. Sets oled_dc to command mode
289                 -- 2. Sends the SetPage Command
290                 -- 3. Sends the Page to be set to
291                 -- 4. Sets the start pixel to the left
                       column
292                 -- 5. Sets oled_dc to data mode
293                 when ClearDC =>
```

```
294                         temp_dc <= '0';
295                         current_state <= SetPage;
296                     when SetPage =>
297                         temp_sdata <= "00100010";
298                         after_state <= PageNum;
299                         current_state <= Transition1;
300                     when PageNum =>
301                         temp_sdata <= "000000" & temp_page;
302                         after_state <= LeftColumn1;
303                         current_state <= Transition1;
304                     when LeftColumn1 =>
305                         temp_sdata <= "00000000";
306                         after_state <= LeftColumn2;
307                         current_state <= Transition1;
308                     when LeftColumn2 =>
309                         temp_sdata <= "00010000";
310                         after_state <= SetDC;
311                         current_state <= Transition1;
312                     when SetDC =>
313                         temp_dc <= '1';
314                         current_state <= after_page_state;
315                     -- End update Page states
316
317                     -- Send character states
318                     -- 1. Sets the address to ASCII value of
                           character with the counter appended
                        to the
319                     --    end
320                     -- 2. Waits a clock cycle for the data
                        to get ready by going to ReadMem and
                        ReadMem2
321                     --    states
322                     -- 3. Send the byte of data given by the
                           ROM
323                     -- 4. Repeat 7 more times for the rest
                        of the character bytes
324                     when SendChar1 =>
325                         temp_addr <= temp_char & "000";
326                         after_state <= SendChar2;
327                         current_state <= ReadMem;
328                     when SendChar2 =>
329                         temp_addr <= temp_char & "001";
330                         after_state <= SendChar3;
```

```
331                         current_state <= ReadMem;
332                     when SendChar3 =>
333                         temp_addr <= temp_char & "010";
334                         after_state <= SendChar4;
335                         current_state <= ReadMem;
336                     when SendChar4 =>
337                         temp_addr <= temp_char & "011";
338                         after_state <= SendChar5;
339                         current_state <= ReadMem;
340                     when SendChar5 =>
341                         temp_addr <= temp_char & "100";
342                         after_state <= SendChar6;
343                         current_state <= ReadMem;
344                     when SendChar6 =>
345                         temp_addr <= temp_char & "101";
346                         after_state <= SendChar7;
347                         current_state <= ReadMem;
348                     when SendChar7 =>
349                         temp_addr <= temp_char & "110";
350                         after_state <= SendChar8;
351                         current_state <= ReadMem;
352                     when SendChar8 =>
353                         temp_addr <= temp_char & "111";
354                         after_state <= after_char_state;
355                         current_state <= ReadMem;
356                     when ReadMem =>
357                         current_state <= ReadMem2;
358                     when ReadMem2 =>
359                         temp_sdata <= temp_dout;
360                         current_state <= Transition1;
361                     -- End send character states
362
363                     -- SPI transitions
364                     -- 1. Set en to 1
365                     -- 2. Waits for spi_ctrl to finish
366                     -- 3. Goes to clear state (Transition5)
367                     when Transition1 =>
368                         temp_spi_en <= '1';
369                         current_state <= Transition2;
370                     when Transition2 =>
371                         if temp_spi_fin = '1' then
372                             current_state <= Transition5;
373                         end if;
```

```
374                     -- End SPI transitions
375
376                     -- Delay transitions
377                     -- 1. Set delay_en to 1
378                     -- 2. Waits for delay to finish
379                     -- 3. Goes to Clear state (Transition5)
380                     when Transition3 =>
381                         temp_delay_en <= '1';
382                         current_state <= Transition4;
383                     when Transition4 =>
384                         if temp_delay_fin = '1' then
385                             current_state <= Transition5;
386                         end if;
387                     -- End Delay transitions
388
389                     -- Clear transition
390                     -- 1. Sets both delay_en and en to 0
391                     -- 2. Go to after state
392                     when Transition5 =>
393                         temp_spi_en <= '0';
394                         temp_delay_en <= '0';
395                         current_state <= after_state;
396                     -- End Clear transition
397
398                     when others =>
399                         current_state <= Idle;
400                 end case;
401             end if;
402         end process;
403
404 end behavioral;
```

Listing 2: `oled_ex.vhd`: demo de OLED adaptado para mostrar A, B y A+B en la primera línea.

Explicación didáctica

- **Interfaz y bloques auxiliares**. La entidad expone el SPI de la OLED (`sdout`, `oled_sclk`, `oled_dc`) y añade las entradas `a[2:0]`, `b[2:0]`. Se instancian tres bloques: `spi_ctrl` (serializa bytes), `delay` (delays en ms) y `ascii_rom` (fuente de 5x8 en bytes).
- **Memoria de pantalla**. `current_screen` es un arreglo (4×16) que representa 4 filas de 16 caracteres ASCII. La FSM recorre `current_screen`

y, para cada carácter, lee 8 bytes de `ascii_rom` y los envía por SPI.

- **Máquina de estados**.
  - `Alphabet` muestra un patrón inicial durante 4 s (`Wait1`).
  - `ClearScreen` limpia y `Wait2` espera 1 s.
  - `HelloWorldScreen` es donde **inyectamos el texto de la suma**: se rellena la fila 0 con `‘‘A=H B=H A+B=H’’`, usando `nibble_to_ascii` para convertir un nibble a ‘0’..‘9’, ‘A’..‘F’.
  - `UpdateScreen` recorre página y columna, y las `SendChar1..8` leen cada byte del glifo desde la ROM y lo envían por SPI.
- **Conversión a ASCII**. La función `nibble_to_ascii` convierte un nibble (4 bits) al código ASCII de un dígito hexadecimal. Así, `'0' & a` y `'0' & b` se muestran en 1 dígito, y `sum4` en otro.
- **Suma encapsulada**. La señal `sum4` proviene del bloque `adder3` (instancia `u_adder3`), lo que evita conflictos de librerías aritméticas dentro de este archivo legado (`std_logic_arith/std_logic_unsigned`).
- **Glifos requeridos**. Para que se vean `‘+’` (0x2B) y `‘=’` (0x3D), esos códigos deben existir en `ascii_rom`. Si no están, se agregan sus 8 bytes y no se toca nada más.
- **Minimalismo**. Fuera de la rama `HelloWorldScreen` y de la instancia de `adder3`, todo el flujo de inicialización, temporización y transmisión SPI permanece intacto.

2.3. Módulo 3: `oled_ctrl` (top: inicialización, multiplexado y ruteo de A, B)

```
1  --
2  -- Written by Ryan Kim, Digilent Inc.
3  -- Modified by Michael Mattioli
4  --
5  -- Description: Top level controller that controls the
       OLED display.
6  --
7
8  library ieee;
9  use ieee.std_logic_1164.all;
10 use ieee.std_logic_arith.all;
```

```
11 use ieee.std_logic_unsigned.all;
12
13 entity oled_ctrl is
14     port (  clk          : in std_logic;
15             rst          : in std_logic;
16             oled_sdin    : out std_logic;
17             oled_sclk    : out std_logic;
18             oled_dc      : out std_logic;
19             oled_res     : out std_logic;
20             oled_vbat    : out std_logic;
21             oled_vdd     : out std_logic;
22             a            : in  std_logic_vector(2 downto
                  0);
23             b            : in  std_logic_vector(2 downto
                  0)
24     );
25 end oled_ctrl;
26
27 architecture behavioral of oled_ctrl is
28
29     component oled_init is
30         port (  clk          : in std_logic;
31                 rst          : in std_logic;
32                 en           : in std_logic;
33                 sdout        : out std_logic;
34                 oled_sclk    : out std_logic;
35                 oled_dc      : out std_logic;
36                 oled_res     : out std_logic;
37                 oled_vbat    : out std_logic;
38                 oled_vdd     : out std_logic;
39                 fin          : out std_logic);
40     end component;
41
42     component oled_ex is
43         port (  clk          : in std_logic;
44                 rst          : in std_logic;
45                 en           : in std_logic;
46                 sdout        : out std_logic;
47                 oled_sclk    : out std_logic;
48                 oled_dc      : out std_logic;
49                 fin          : out std_logic;
50                 -- NUEVO:
```

```
51                  a           : in  std_logic_vector(2
                       downto 0);
52                  b           : in  std_logic_vector(2
                       downto 0)
53          );
54      end component;
55
56      type states is (Idle, OledInitialize, OledExample,
           Done);
57
58      signal current_state     : states := Idle;
59
60      signal init_en           : std_logic := '0';
61      signal init_done         : std_logic;
62      signal init_sdata        : std_logic;
63      signal init_spi_clk      : std_logic;
64      signal init_dc           : std_logic;
65
66      signal example_en        : std_logic := '0';
67      signal example_sdata     : std_logic;
68      signal example_spi_clk   : std_logic;
69      signal example_dc        : std_logic;
70      signal example_done      : std_logic;
71
72  begin
73
74      Initialize: oled_init port map ( clk,
75                                       rst,
76                                       init_en,
77                                       init_sdata,
78                                       init_spi_clk,
79                                       init_dc,
80                                       oled_res,
81                                       oled_vbat,
82                                       oled_vdd,
83                                       init_done );
84
85      Example: oled_ex port map ( clk,
86                                  rst,
87                                  example_en,
88                                  example_sdata,
89                                  example_spi_clk,
90                                  example_dc,
```

```
91                                  example_done,
92                                  -- NUEVO:
93                                  a,
94                                  b );
95
96      -- MUXes to indicate which outputs are routed out
           depending on which block is enabled
97      oled_sdin <= init_sdata when current_state =
           OledInitialize else example_sdata;
98      oled_sclk <= init_spi_clk when current_state =
           OledInitialize else example_spi_clk;
99      oled_dc   <= init_dc    when current_state =
           OledInitialize else example_dc;
100     -- End output MUXes
101
102     -- MUXes that enable blocks when in the proper
           states
103     init_en    <= '1' when current_state =
           OledInitialize else '0';
104     example_en <= '1' when current_state = OledExample
              else '0';
105     -- End enable MUXes
106
107     process (clk)
108     begin
109         if rising_edge(clk) then
110             if rst = '1' then
111                 current_state <= Idle;
112             else
113                 case current_state is
114                     when Idle =>
115                         current_state <= OledInitialize;
116                     -- Go through the initialization
                           sequence
117                     when OledInitialize =>
118                         if init_done = '1' then
119                             current_state <= OledExample
                                   ;
120                         end if;
121                     -- Do example and do nothing when
                           finished
122                     when OledExample =>
123                         if example_done = '1' then
```

```
124                                 current_state <= Done;
125                             end if;
126                         -- Do nothing
127                         when Done =>
128                             current_state <= Done;
129                         when others =>
130                             current_state <= Idle;
131                     end case;
132                 end if;
133             end if;
134         end process;
135
136 end behavioral;
```

Listing 3: `oled_ctrl.vhd`: top que inicializa la OLED y enruta los switches `a,b` hacia `oled_ex`.

Explicación didáctica

- **Rol del top**. `oled_ctrl` es el punto de entrada del diseño: expone los pines físicos hacia la OLED y (ahora) los puertos `a[2:0]`, `b[2:0]` que provienen de los *switches* de la ZedBoard. Este archivo es el que se asocia al XDC.
- **Jerarquía y ruteo**. Instancia dos bloques:
  1. `oled_init`: ejecuta la secuencia de encendido de la SSD1306 (tensión, reset, comandos base).
  2. `oled_ex`: el demo que *presenta* contenido; en esta práctica se modificó para mostrar A, B y $A{+}B$.

  Los puertos `a` y `b` se **pasan directo** a `oled_ex` (no hay lógica adicional aquí).
- **Máquina de estados mínima**. Tres fases:
  1. `OledInitialize`: habilita `oled_init` hasta que `init_done='1'`.
  2. `OledExample`: habilita `oled_ex` hasta que `example_done='1'`.
  3. `Done`: estado estable final.
- **Multiplexado de salidas**. Mientras se inicializa, las señales a la pantalla (`oled_sdin`, `oled_sclk`, `oled_dc`) provienen de `oled_init`; después, provienen de `oled_ex`. Este MUX garantiza que no haya *batalla* de controladores sobre el bus SPI ni el pin D/C.

- **Dónde "vive" la suma**. La aritmética se encapsula en `adder3` (instanciado dentro de `oled_ex`); el top `oled_ctrl` solo enruta `a,b`. Así evitamos cambiar paquetes o introducir operaciones aritméticas en el código legado del top.

3. Asignación de pines y señales

En esta práctica el *top* `oled_ctrl` expone las **entradas** desde los *switches* (operandos A y B), el **reloj** y el **reset**; y enruta las **salidas** hacia la pantalla OLED (interfaz SPI y control). A continuación se listan, de forma didáctica, las señales, su dirección y el pin físico al que deben conectarse en la ZedBoard.

Entradas (reloj, reset y operandos)

Señal	Dirección	Pin físico
`clk`	*in*	Y9
`rst`	*in*	P16
`a[2]`	*in*	H22
`a[1]`	*in*	G22
`a[0]`	*in*	F22
`b[2]`	*in*	H18
`b[1]`	*in*	H19
`b[0]`	*in*	F21

Salidas hacia la OLED (SPI + control)

Señal	Dirección	Pin físico
`oled_sdin`	*out*	AA12
`oled_sclk`	*out*	AB12
`oled_dc`	*out*	U10
`oled_res`	*out*	U9
`oled_vbat`	*out*	U11
`oled_vdd`	*out*	U12

Notas prácticas

- Los bits se enumeran de mayor a menor: `a[2]` y `b[2]` son los MSB (bit más significativo); `a[0]` y `b[0]` los LSB.

- El reloj del sistema (`clk`) es de 100 MHz; el reset `rst` es síncrono y activo en alto tal como se usa en el código.
- Verifica la orientación de los *switches* en la tarjeta para que el valor visualizado en la OLED coincida con el patrón esperado (MSB–LSB).

4. Resultados

En la Fig. 1 y Fig. 2 se muestran dos capturas del funcionamiento: primero la visualización del patrón de *alfabeto* (pantalla de prueba tras la inicialización) y, a continuación, la visualización del sumador A, B y $A{+}B$ en la primera línea de la OLED.

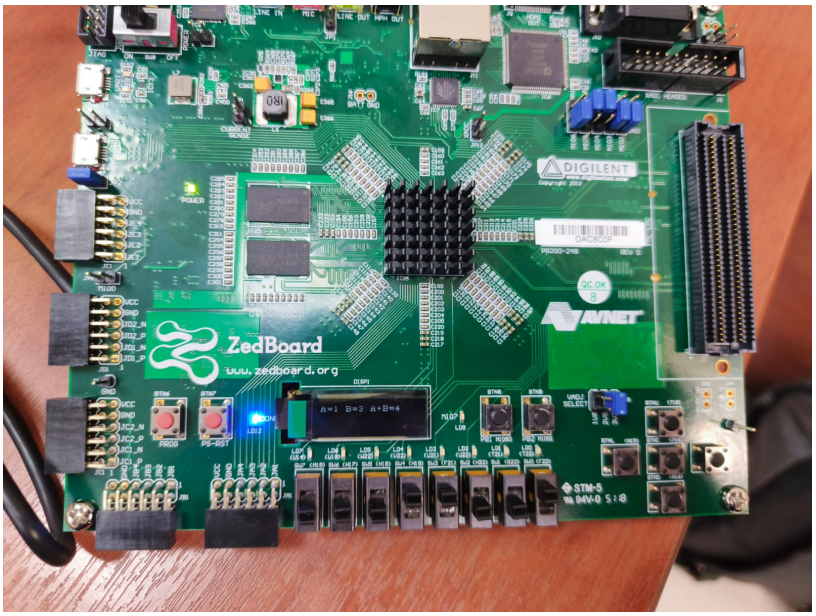


Figura 1: Pantalla del sumador Escenario 1.

Actualización de la visualización al cambiar A o B. Para ver reflejado un nuevo valor de la operación en la OLED, es necesario **hacer que la FSM vuelva a construir y enviar el *frame***. En el diseño base, esto se logra *cambiando de estado* (forzando un nuevo recorrido por `HelloWorldScreen` → `UpdateScreen`). Operativamente, el flujo típico es:

1. Volver a la pantalla de *alfabeto* (cambio de estado).
2. Regresar a la pantalla del *sumador* (nuevo cambio de estado).

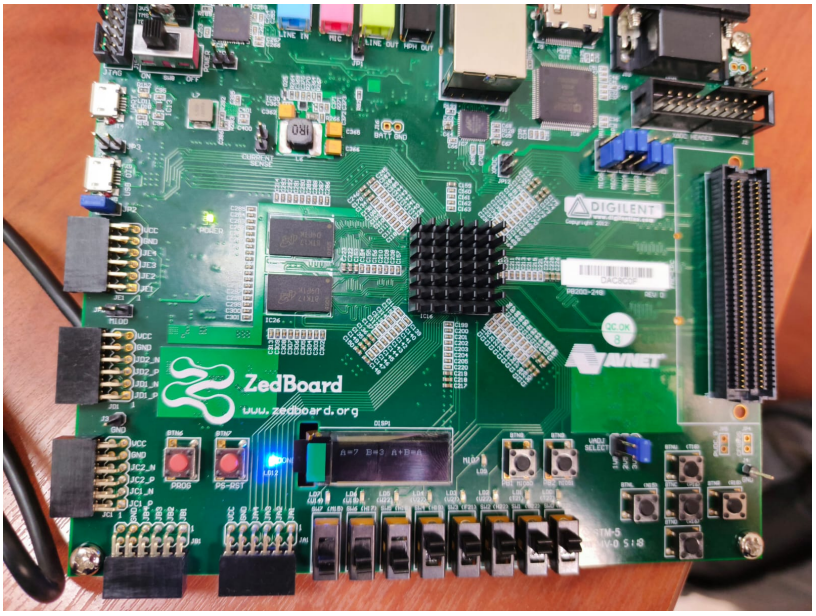


Figura 2: Pantalla del sumador Escenario 2.

Con ese ciclo, los caracteres se regeneran y la suma $A+B$ se actualiza correctamente.